

Programmierung mit ANSI-C

Skript nach einer Vorlesungsausarbeitung von Martin Lowes

Überarbeitet von Carsten Damm und Henrik Brosenne

Die Inhalte dieses Skripts sind urheberrechtlich
geschützt und dürfen nur zu eigenen
Studienzwecken verwendet werden.

Jede weitere Verbreitung ist unzulässig!

Georg-August-Universität
Göttingen

Vorwort

Der vorliegende Text ist auf Grundlage einer Vorlesungsausarbeitung von Martin Lowes entstanden, der bis zu seinem Tode 1999 viele Jahre lang Programmierkurse an der Georg-August-Universität in Göttingen durchgeführt hat. Die didaktisch gelungene Struktur und der Inhalt des Urtexts sind weitgehend erhalten geblieben ebenso wie die meisten Programmbeispiele. In die Aktualisierung, Ergänzung, Korrektur, Vereinheitlichung und das Layout des Materials wurde viel Mühe gesteckt, so dass es nun sehr gut sowohl als Begleitmaterial zu Programmierkursen als auch später zum Nachschlagen dienen kann. Ich danke Herrn Johannes Löwe, der den Text (unter Verwendung alter Quellen) einheitlich und sehr sorgfältig mit L^AT_EX gesetzt hat, aber auch zahlreiche inhaltliche Verbesserungen, Textvorschläge und Korrekturen einbrachte sowie eigene Programmbeispiele beisteuerte. Ich danke auch dem Institut für Numerische und Angewandte Mathematik der Universität Göttingen für die Ermöglichung dieser Arbeiten.

Jedes Kapitel entspricht durchschnittlich im Umfang etwas mehr als einer eineinhalbstündigen Vorlesung, wobei der Schwierigkeitsgrad von Kapitel zu Kapitel steigt. Lässt man einige Abschnitte aus, so kann man in einem Drei-Wochen-Kurs einen Großteil des hier gebotenen Stoffs vermitteln. Die Auslassungen können sich interessierte Teilnehmer im Selbststudium erarbeiten.

C ist eine Programmiersprache, die einer weltweiten Normierung unterliegt. Der aktuelle Standard ist ISO/IEC 9899:1999 (kurz: C99). Jedoch unterstützen nicht alle modernen C-Compiler ohne weiteres alle dort festgeschriebenen Neuerungen gegenüber älteren Versionen des Standards. Sehr weit verbreitet und von den meisten modernen Compilern sehr gut unterstützt ist dagegen der Standard ISO/IEC 9899:1990 (fast identisch zum sogenannten ANSI-Standard (C89) und darum im weiteren so genannt). Es wurde daher Wert darauf gelegt, dass alle Konzepte gründlich und im Sinne einer strengen Auslegung des ANSI-Standards vermittelt werden, ohne die Neuerungen aus C99 zu besprechen. Die noch unzureichende Compiler-Unterstützung würde Anfängern wohl manche Verwirrung bereiten. Die Beschreibung der Programmiersprache erfolgt in diesem Text weitestgehend Betriebssystem-unabhängig. Beschreibung von Compiler-Aufrufen dagegen beziehen sich ausschließlich auf den GNU C-Compiler.

Zielgruppe für den Text sind in erster Linie Studierende, die sauberes Programmieren in ANSI-C erlernen wollen (oder müssen), aber auch Autodidakten und Fortgeschrittene, die sich über einige Aspekte genauer informieren wollen. An Umfang und Präzision des Standards selbst kann und soll der Text selbstverständlich nicht heranreichen.

Die Programmbeispiele im Text sind natürlich nicht ausreichend, um wirklich Programmieren zu lernen. Sie illustrieren nur den einen oder anderen Aspekt der Programmiersprache und sind in keinem Falle Praxisbeispiele. Vielmehr muss das aufmerksame Lesen durch praktische Übungen am Computer ergänzt werden, die vom Ausprobieren der Programmbeispiele und das Herumspielen damit über die Lösung kleiner Übungsaufgaben zu wirklich interessanten und nützlichen Programmen reichen.

Anfängern sei gesagt: Verzagen Sie nicht! Aller Anfang ist schwer, aber wenn der Knoten geplatzt ist, macht C-Programmieren wirklich Spaß. Und noch ein Wort für Fortgeschrittene: Auch wenn Sie schon die eine oder andere ähnliche Programmiersprache kennen und es Ihnen am Anfang zu langsam voran geht — Unterschätzen Sie die Feinheiten und Fallstricke der Programmiersprache C nicht, es wird schon noch interessant!

Viel Erfolg!!

Vorwort zur zweiten Auflage

Entsprechend dem Wunsch vieler Hörer wurde für schnelles Nachschlagen ein kurzer Anhang zur Standardbibliothek eingefügt. Johannes Löwe hat das Skript nochmals durchgesehen und die zahlreichen weiteren Verbesserungsvorschläge und Korrekturen eingearbeitet. Außerdem hat er das Stichwortverzeichnis überarbeitet und ergänzt. Herzlichen Dank für diese Arbeit! Danke natürlich auch an die Teilnehmer, die die Verbesserungsvorschläge eingebracht haben.

Carsten Damm

Februar 2007

Vorwort zur dritten Auflage

Kleinere Korrekturen wurden vorgenommen und einige Tippfehler beseitigt. Der Inhalt ist weitgehend unverändert, insbesondere die Abfolge und Aufteilung der Kapitel ist gleich geblieben.

Henrik Brosenne

Februar 2010

Inhaltsverzeichnis

1	Einleitung	1
1.1	Die Geschichte von C	1
1.2	Schritte der Programmentwicklung	2
1.3	Der Zeichensatz von C	3
1.4	Der Aufbau von C-Programmen	4
1.5	Literatur	7
2	Standarddatentypen, Konstanten und Variablen	9
2.1	Interne Darstellungen	9
2.2	Wertebereiche	10
2.3	Ganzzahlige Typen	10
2.4	Der Typ <code>char</code>	12
2.5	Zeichenkettenkonstanten	14
2.6	Gleitkommatypen	14
2.7	Aufzählungskonstanten	15
2.8	Benannte Konstanten	15
2.9	Variablen	16
2.10	Namen	17
2.11	Typnamen	19
3	Beispielprogramme	21
3.1	Formatierte Ein-/Ausgabe	21
3.2	Formatierte Ausgabe	22
3.3	Formatierte Eingabe	24
3.4	Schleifen	25
3.5	Wertzuweisung	26
3.6	Zeilenorientierte Verarbeitung	27
3.7	Lesen bis zum Ende	28
3.8	Alternativen	29
3.9	Felder	30
4	Ausdrücke und Operatoren	33
4.1	Aufbau von Ausdrücken	33
4.2	Die Wertzuweisung	33
4.3	Arithmetische Operatoren	34
4.4	Inkrementierung und Dekrementierung	35
4.5	Vergleichsoperatoren	36
4.6	Logische Operatoren	37
4.7	Zusammengesetzte Zuweisungen	38
4.8	Bedingte Ausdrücke	38
4.9	Der Kommaoperator	39
4.10	Priorität der Operatoren	39
4.11	Nebeneffekte	41
4.12	Typumwandlung	42
4.13	Castoperatoren	44

4.14	Der <code>sizeof</code> -Operator	45
5	Zeichen und Strings	47
5.1	Stringvariablen und -konstanten	47
5.2	Arbeiten mit Strings	48
5.3	Ein-/Ausgabe von Zeichen	48
5.4	Ein-/Ausgabe von Strings	50
5.5	Klassifizierung von Zeichen	51
6	Steuerung des Programmablaufs	55
6.1	Anweisungen und Blöcke	55
6.2	Die <code>if</code> -Anweisung	55
6.3	Die <code>switch</code> -Anweisung	58
6.4	Die <code>while</code> -Schleife	60
6.5	Die <code>do</code> -Schleife	60
6.6	Die <code>for</code> -Schleife	62
6.7	Sprünge	64
6.8	Die <code>break</code> -Anweisung	65
6.9	Die <code>continue</code> -Anweisung	66
6.10	Beispiel	66
7	Funktionen	69
7.1	Motivation	69
7.2	Vereinbarung von Funktionen	69
7.3	Funktionswerte	71
7.4	Aufruf von Funktionen	71
7.5	Parameter und Argumente	72
7.6	Felder als Parameter	73
7.7	Das Attribut <code>const</code>	75
7.8	Prototypen	75
7.9	Rekursion	75
7.10	Beispiel: Türme von Hanoi	77
7.11	Beispiel: Quicksort	78
7.12	Beispiel: Potenzen	81
8	Strukturierung von Programmen	83
8.1	Gültigkeitsbereiche von Namen	83
8.2	Interne und externe Größen	85
8.3	Das Modulkonzept	87
8.4	Separate Compilation, <code>make</code>	90
8.5	Lokale und globale Größen	91
8.6	Deklarationen und Definitionen	92
8.7	Statische und automatische Variablen	94
8.8	<code>register</code> und <code>volatile</code>	96
9	Übersicht über die Standardbibliothek	99
9.1	Headerdateien	99
9.2	Mathematische Funktionen	100
9.3	Fehlerbehandlung	101
9.4	Elementare Typen	102

9.5	Diverse Hilfsroutinen	102
9.6	Termine und Zeiten	103
10	Der Präprozessor	105
10.1	Überblick	105
10.2	Die Direktive <code>#include</code>	106
10.3	Makros (<code>#define</code>)	106
10.4	Bedingte Compilation (<code>#if</code> , <code>#elif</code> , <code>#else</code>)	107
10.5	Weitere Möglichkeiten (<code>#ifndef</code> , <code>#undef</code>)	109
10.6	Makro-Definition im Compileraufruf	110
11	Felder	111
11.1	Rückblick	111
11.2	Vereinbarung von Feldern	111
11.3	Anordnung von Feldern im Speicher	112
11.4	Felder als Parameter	113
11.5	Initialisierung von Feldern	116
12	Zeiger	119
12.1	Adressen und Zeiger	119
12.2	Zeiger als Funktionsparameter (I)	120
12.3	Zeigervariablen	121
12.4	Zeiger und <code>const</code>	122
12.5	Zeiger und Felder (I)	123
12.6	Zeigerarithmetik	124
12.7	Operationen mit Zeigern	125
12.8	Zeiger als Parameter (II)	127
12.9	Probleme mit Zeigern	128
12.10	Zeiger und Felder (II)	130
12.11	Zeigervektoren	131
12.12	Zeiger auf Zeiger	132
12.13	Zeiger als Funktionswerte	133
12.14	Parameter des Hauptprogramms	134
12.15	Zeiger auf Funktionen	136
12.16	Kopieren und umspeichern von Speicherblöcken	139
13	Speicher auf dem Heap	141
13.1	Speichertypen	141
13.2	Funktionen zur Heapverwaltung	142
13.3	Bereitstellung von Speicher	142
13.4	Beispiel: Speichern von Zahlen	143
13.5	Speicherfreigabe	145
13.6	Matrizen auf dem Heap	146
14	Strukturen	149
14.1	Vereinbarung von Strukturen	149
14.2	Operationen mit Strukturen	150
14.3	Schachtelung strukturierter Typen	152
14.4	Zeiger auf Strukturen	153
14.5	Strukturen auf dem Heap	155

14.6	Datum und Uhrzeit	155
14.7	Verkettete Listen	156
14.8	Verbunde	157
15	Probleme der Rechnerarithmetik	159
15.1	Gleitkomma-Ausnahmebehandlung	159
15.2	Überlauf bei ganzen Zahlen	160
15.3	Normalisierte Gleitkomma-Darstellungen	160
15.4	Assoziativ- und Kommutativgesetz	161
15.5	Rundungsfehler	162
15.6	Auslöschung	163
16	Ein-/Ausgabe	165
16.1	Verarbeitung von Dateien	165
16.2	Formatierte und binäre Ein-/Ausgabe	167
16.3	Ausgabeformate	168
16.4	Eingabeformate	171
16.5	Funktionen zur Ein-/Ausgabe	174
16.6	Vorausschau beim Lesen	174
A	Einführung in UNIX	177
A.1	Grundlagen	177
A.2	Ein- und Ausloggen, Passwort	177
A.3	Hilfen	178
A.4	Das Dateisystem	178
A.5	Dateiverwaltung	179
A.6	Metazeichen	182
A.7	Auflisten von Datei-Inhalten	182
A.8	Umleitung von Ein- und Ausgabe	183
A.9	Editieren	183
A.10	Übersetzen und Binden	183
B	Die C-Standardbibliothek	185
	Abbildungsverzeichnis	191
	Tabellenverzeichnis	191
	Literaturverzeichnis	193
	Stichwortverzeichnis	195

Kapitel 1

Einleitung

1.1 Die Geschichte von C

Die Programmiersprache C wurde 1972 von Dennis M. Ritchie entwickelt. Der Anlass war eine Rechner-unabhängige Implementation des Betriebssystems UNIX. Dieses Betriebssystem ist zu über 90 % in C geschrieben.

Viele der wichtigsten Ideen von C stammen aus der Sprache BCPL (*Basic Combined Programming Language*), die Martin Richards 1967 entwickelt hat. Diese Sprache wurde von Ken Thomson 1970 für das erste UNIX-System zur Sprache B weiterentwickelt, die schließlich zu C erweitert wurde.

Viele Jahre lang galt die erste Auflage der Sprachbeschreibung *Programmieren in C* von Brian W. Kernighan und Dennis M. Ritchies quasi als „Standard“ für C. Diese Sprachbeschreibung war aber weder vollständig noch exakt; für den Anwender reichte sie zwar aus, für einen Compilerbauer fehlte jedoch vieles. Deshalb richtete 1983 ANSI (*American National Standards Institute*) eine Kommission ein, die eine moderne, umfassende Definition von C erstellen sollte. Das Resultat, der *ANSI-Standard* oder *ANSI-C*, wurde Ende 1988 vorgelegt und ein Jahr später veröffentlicht.

Das oben erwähnte Buch von Kernighan und Ritchie erscheint in seiner zweiten Ausgabe, die den ANSI-Standard als Grundlage beschreibt, und gilt als das Buch über C:

Brian W. Kernighan & Dennis M. Ritchie:
The C programming language, Second Edition
Prentice Hall, Englewood Cliffs N.Y. (1988)

1994 wurden die ersten Ergänzungen zum ANSI-Standard (ISO C Amendment 1) eingeführt. 1999 wurde der Standard einer Überarbeitung unterzogen. Dabei wurde auf Abwärtskompatibilität geachtet (ANSI C-Programme nach dem Standard von 1989 genügen weitestgehend auch der Neufassung) und einige von Compiler-Herstellern vorgeschlagene Erweiterungen wurden standardisiert.

Der Standard von 1989 beruht in weiten Teilen auf der ursprünglichen Sprachbeschreibung von Kernighan/Ritchie. An den Grundelementen der Sprache hat der Standard nicht sehr viel geändert. Allerdings gibt es durchaus auch wesentliche Neuerungen:

- Die Syntax zur Deklaration und Definition von Funktionen wurde so erweitert, dass der Compiler eventuelle Widersprüche zwischen den Parametern in Deklaration bzw. Definition einer Funktion und den Argumenten in ihren Aufrufen erkennen kann.
- Der Standard legt fest, welchen Umfang die *Standardbibliothek* haben muss. Diese

Bibliothek enthält Funktionen zur Ein-/Ausgabe, zum Zugriff auf das Betriebssystem, zur Manipulation von Zeichenketten, usw.

Der Vorteil dieser Definition ist, dass jedes Standardprogramm, das zum Zugriff auf Betriebssystem und/oder Hardware des Rechners ausschließlich Funktionen aus dieser Bibliothek verwendet, *portabel* ist, d.h. es lässt sich auf jedem beliebigen Rechner, auf dem ein Standardcompiler mit Standardbibliothek zur Verfügung steht, compilieren.

- Die Regeln für numerisches Rechnen wurden grundlegend überarbeitet.

C gilt als höhere Programmiersprache. Es enthält die üblichen Sprachkonstrukte zur Steuerung des Kontrollflusses (Entscheidungen, Schleifen) und bietet Hilfsmittel zur Strukturierung und Modularisierung von Programmen. Gleichzeitig ist C eine maschinennahe Sprache, d.h. man kann in C, ähnlich wie in einem Assembler, Bits, Bytes und Adressen manipulieren.

Ein Hauptvorwurf der Kritiker von C ist, dass C eine zu hohe Disziplin des Programmierers voraussetzt, damit seine Programme nicht völlig unleserlich und instabil geraten. Das ist aber eine fast zwangsläufige Konsequenz der Intentionen für die Entwicklung von C: Die Zielgruppe waren zunächst Systemprogrammierer, also erfahrene Programmierer, die eine Möglichkeit erhalten sollten, auch maschinennahe Programme zum einen maschinenunabhängig und zum anderen kurz und prägnant zu formulieren.

1.2 Schritte der Programmentwicklung

Die Entwicklung eines Programms erfolgt in mehreren Schritten, unabhängig davon, mit welcher Programmiersprache man arbeitet.

1. Die Aufgabe muss sauber definiert werden. Hierzu gehört: Welche Daten stehen in welcher Form zur Verfügung? Welche Resultate möchte man haben?
2. Es muss ein *Algorithmus* zur Lösung der Aufgabe gesucht und formuliert werden.

Allgemeine Regeln, wie man zu einem Problem einen geeigneten Algorithmus findet, kann es nicht geben; allerdings gibt es Hilfsmittel, wie man Algorithmen übersichtlich formulieren kann. Eines davon ist ein *Flussdiagramm*.

Die Formulierung von Algorithmen erfolgt, wie die Aufgabenstellung selbst, abseits vom Rechner und auch unabhängig von der zu verwendenden Programmiersprache.

3. Der Algorithmus wird *codiert*, d.h. seine Einzelschritte werden in Anweisungen einer speziellen Programmiersprache übertragen und *erfasst*, d.h. in den Rechner eingegeben und dort z.B. auf der Festplatte gespeichert. Hierzu dienen i.a. spezielle Programme, die *Editor* genannt werden. Sie helfen dem Entwickler meist durch automatische Formatierung des Quelltextes, *Syntaxeinfärbung* – d.h. Schlüsselworte oder Kommentare werden in besonderen Farben hervorgehoben, automatische Vervollständigung von Namen oder zeigen z.B. die möglichen Parameter einer Funktion an.

Ein Programm in dieser Form wird als *symbolisches Programm* oder auch als *Quellcode* bzw. *Quelltext* bezeichnet.

4. Das gespeicherte symbolische Programm wird mit einem *Compiler* übersetzt.

In einem neu erfassten Programm wird der Compiler i.a. mehr oder minder viele formale Fehler finden – diese müssen zunächst mit dem Editor korrigiert werden. Die Folge editieren/übersetzen muss so lange wiederholt werden, bis das Programm keine formalen Fehler mehr enthält.

Wenn der Compiler keine Fehler entdeckt, erzeugt er ein *Objekt-Programm*.

5. Das Objekt-Programm wird *gebunden*; dieses geschieht erneut durch ein spezielles Programm, den *Linker*. Der Linker fügt das eigene Programm mit den benötigten Routinen aus den Programm-Bibliotheken zu einem *ausführbaren Programm* („Binary“) zusammen.

Auch dabei können Fehler auftreten, nämlich dass der Linker benötigte Routinen nicht findet.

6. Das fertige Programm kann ausgeführt werden.

Allerdings: Man kann nicht davon ausgehen, dass jedes Programm beim ersten Versuch korrekt arbeitet. Jetzt heißt es also i.a., die logischen Fehler im Quell-Programm zu suchen, sie mit dem Editor zu korrigieren, usw. – so lange, bis das Programm korrekt arbeitet.

Manche Entwicklungssysteme koppeln die Schritte (3) und (4) oder sogar die Schritte (3) bis (5) so eng aneinander, dass sie kaum noch als Einzelschritte zu erkennen sind. In der Sache ändert das am Ablauf allerdings nichts und man darf auch in diesen Fällen nicht aus dem Auge verlieren, dass verschiedene Schritte ausgeführt werden.

1.3 Der Zeichensatz von C

Es ist wichtig zu wissen, welche Zeichen in einer Quelltextdatei stehen dürfen, damit der Compiler sie verarbeiten kann. Der ANSI-Standard schreibt folgende Zeichen vor:

Der Zeichensatz von C umfasst 91 graphische Zeichen (d.h. Zeichen, die durch Drucker-schwärze auf dem Papier unmittelbar dargestellt werden können) und weitere Zeichen. Die Zeichen lassen sich in 5 Gruppen einteilen:

- 26 Großbuchstaben des (englischen) Alphabets

A	B	C	D	E	F	G	H	I	J	K	L	M
N	O	P	Q	R	S	T	U	V	W	X	Y	Z

- 26 Kleinbuchstaben des (englischen) Alphabets

a	b	c	d	e	f	g	h	i	j	k	l	m
n	o	p	q	r	s	t	u	v	w	x	y	z

- 10 Ziffern

0	1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---	---

- 29 Sonderzeichen

!	"	#	%	&	'	(	)	*	+	,	-	.	/	:
;	<	=	>	?	[	\	]	^	_	{		}	~	

- Weitere Zeichen, die keine *graphischen Zeichen* sind. Dazu gehören das Leerzeichen (in Texten wie diesen durch `_` gekennzeichnet) und Steuerzeichen: zwei Tabulator-Zeichen (horizontal und vertikal), ein Seitenvorschub-Zeichen und die Zeilenende-Kennzeichnung. Diese Zeichen werden kollektiv als *white spaces* bezeichnet.

Die Zeilenende-Kennzeichnung kann implementationsabhängig aus mehreren Zeichen bestehen. UNIX/Linux-Textdateien verwenden nur das linefeed-Zeichen als Zeilenendezeichen. DOS/Windows-Textdateien benutzen dagegen eine Kombination aus linefeed und carriage return (Repositionierungszeichen). Apple-Textdateien wiederum benutzen nur das carriage return.

C-Programme können jedoch während der Ausführung auch andere Zeichen *verarbeiten*. Eine Benutzereingabe kann also durchaus auch die deutschen Umlaute oder das ß enthalten, obwohl der Quelltext diese Zeichen nicht enthalten darf. Zur Darstellung dieser Zeichen im Quelltext z.B. in Zeichenkettenkonstanten gibt es spezielle Zeichenkombinationen, die sogenannten Escapesequenzen.

1.4 Der Aufbau von C-Programmen

Ein C-Programm besteht, formal betrachtet, aus einer Ansammlung von *Funktionen*. Genau eine dieser Funktionen ist ausgezeichnet durch den Namen `main` und realisiert das *Hauptprogramm*. Alle anderen Funktionen sind *Unterprogramme* und müssen andere Namen tragen. Das kleinste Programm besteht nur aus der Funktion `main`. Fehlt sie, gibt der Linker eine Fehlermeldung aus.

Alle Funktionen sind formal völlig gleichberechtigt. Insbesondere heißt das, dass zwar alle Funktionen in beliebiger Reihenfolge angegeben werden können, nicht jedoch ineinander geschachtelt.

Das Schema eines C-Programms sieht so aus:

direktiven für den präprozessor
globale deklarationen

```
... funkt1(...) {
    lokale deklarationen
    anweisungsfolge
}
```

...

```
... funktN(...) {
    lokale deklarationen
    anweisungsfolge
}
```

```
int main(...) {
    lokale deklarationen
    anweisungsfolge
}
```

`main` ist, wie bereits gesagt, das Hauptprogramm, d.h. die Ausführung des gesamten Programms beginnt am Anfang von `main`. Die Funktionen `funkt1`, ..., `funktN` können vom Hauptprogramm oder auch von den Funktionen aufgerufen werden. Insbesondere ist auch *Rekursion* erlaubt, d.h. eine Funktion darf sich selbst aufrufen.

C unterscheidet bei Unterprogrammen formal nicht zwischen Prozeduren und Funktionen, wie etwa Pascal. Mit Funktionen sind Unterprogramme gemeint, die nach dem Aufruf einen Funktionswert zurückgeben (vergleichbar mit einer mathematischen Funktion), während Prozeduren keinen Wert zurückgeben und allein durch ihre Nebeneffekte wirken. In C gibt es nur Funktionen – wir werden allerdings sehen, dass auch C die Unterscheidung in der Sache durchaus kennt.

Sehen wir uns nun ein erstes vollständiges Programm an. Es soll einfach nur den Text

`Unser erstes Beispiel!`

auf der Standardausgabe ausgeben. Aussehen kann es so:

```

/*****\
 * Unser erstes Beispiel                      *
 \*****/

#include <stdio.h>                          /* Praeprozessordirektive */

/*= Hauptprogramm =====*/
int main(void) {
    printf("Unser erstes Beispiel!\n");      /* Textausgabe      */
    return 0;                               /* fertig           */
}

```

Quelltext 1.1: Unser Erstes Beispiel

Sehen wir uns die Details näher an:

- Das Programm beginnt mit einem *Kommentar*: Zeichenfolgen, die mit `/*` eingeleitet und mit `*/` beendet werden, ignoriert der Compiler. Es ist offensichtlich, dass Kommentare nicht geschachtelt werden können, da das erste Auftreten von `*/` den Kommentar beendet, egal wie oft `/*` bisher gefunden wurde.

Man unterscheidet zwei Arten von Kommentaren:

- *Kommentarblöcke* erstrecken sich über die ganze Breite der Zeilen. Jede Quelldatei sollte mit einem solchen Block beginnen, in dem ihr Inhalt kurz erläutert und in der Regel zumindest auch der Autor und das Datum der letzten Änderung genannt werden. In den weiteren Quelltextbeispielen wird der einleitende Kommentarblock nicht abgedruckt. Enthält die Quelldatei mehrere Funktionen, so sollte jede Funktion durch einen eigenen Kommentarblock eingeleitet werden. Für die Umrandungen sollte man dann Zeichen mit unterschiedlichem Schwärzungsgrad verwenden.
- *Inline-Kommentare* dienen zur Erläuterung des Ablaufs. Es sollte nicht jede einzelne Zeile kommentiert werden, man sollte aber auch nicht zu sparsam mit Kommentaren sein. Dabei sollte man zu beschreiben versuchen, „warum“ etwas geschieht, nicht „was“ geschieht – das sieht man ohnehin! Bei den Inline-Kommentaren sollte man auf eine strikte vertikale Trennung zwischen Code und Kommentar achten.
- Die Routinen zur Ein-/Ausgabe sind, wie bereits angesprochen, nicht direkt als Bestandteil von C definiert. Die Standardbibliothek enthält jedoch unter anderem entsprechende Funktionen, zum Beispiel `printf` für die Ausgabe.

Ohne weiteres kennt der Compiler die Funktionen der Standardbibliothek nicht, so dass sie ihm bekanntgemacht werden müssen. Da es offenbar zu mühsam (und fehleranfällig) wäre, die entsprechenden Deklarationen jeweils explizit hinzuschreiben, gehören zur Standardbibliothek eine Reihe von Standard-Headerdateien, die die entsprechenden Deklarationen enthalten.

Der Zugriff auf diese Headerdateien erfolgt letztlich in zwei Schritten:

1. Zunächst bearbeitet ein *Präprozessor*, dessen Funktionsumfang auch durch den Standard festgelegt ist, das Programm. Ihm gilt die Präprozessor-Direktive

```
#include <stdio.h>
```

Der Präprozessor ersetzt diese Zeile durch den Inhalt einer Datei `stdio.h` – und das ist gerade die Datei, die die Deklaration der Funktion `printf` enthält.

Das Nummernzeichen (**#**) muss übrigens – abgesehen von eventuellen Leer- und Tabulatorzeichen – das erste Zeichen in der Zeile sein, damit der Präprozessor sich „zuständig fühlt“. Weitere *Präprozessor-Direktiven* werden wir später kennenlernen.

2. Wenn der Präprozessor seine Arbeit beendet hat, beginnt die eigentliche Übersetzung.
- Da das Programm nur aus einer Funktion, nämlich **main**, besteht, muss diese Funktion jetzt folgen.

Vor dem Funktionsnamen steht der Typ des Funktionswertes, den die Funktion liefert. Für **main** ist festgelegt, dass der Funktionswert ganzzahlig ist; C bezeichnet das mit **int**.

Dem Funktionsnamen folgt, in runde Klammern eingeschlossen, die Parameterliste, die die Kommunikation zwischen einer Funktion und ihrer Umwelt ermöglicht. Wenn eine Funktion, wie hier **main**, keine Parameter besitzt, ist zwischen den Klammern das Schlüsselwort **void** einzutragen.

Achtung: In der Literatur findet man oft **main()** statt **main(void)**. Dieses entspricht *nicht* dem Standard. Das Weglassen des Funktionstyps hingegen ist erlaubt, wenn auch unschön: Immer wenn der Typ einer Funktion nicht explizit angegeben ist, ergänzt der Compiler **int**.

- Der Rumpf einer Funktion, d.h. die Anweisungen, die bei Aufruf der Funktion ausgeführt werden, wird in geschweifte Klammern eingeschlossen.

Der Übersichtlichkeit halber ist es üblich, die Anweisungen innerhalb eines Rumpfes einzurücken. Editoren, die speziell zum Erfassen von Quelltexten entwickelt wurden, rücken Blöcke meist automatisch ein.

- Die erste Anweisung in unserem Beispiel ist der Aufruf der Bibliotheksfunktion **printf**. Diese Funktion schreibt die in Gänsefüßchen stehende Zeichenkette auf den Bildschirm.

In der Zeichenkette steht die Zeichenkombination **\n**. Sie gilt in C als *ein* Zeichen und bewirkt bei ihrer Ausgabe den Übergang zu einer neuen Zeile. Es handelt sich um die Umschreibung des newline-Zeichens. Die Notwendigkeit einer solchen Umschreibung ergibt sich aus der Tatsache, dass der Compiler schließende Gänsefüßchen in der gleichen Zeile erwartet. Nicht zulässig ist es also, einen Zeilenumbruch (durch ENTER) direkt in die Zeichenkette einzufügen. Beim Editieren würde das so aussehen:

```
printf("Unser erstes Beispiel!
");
```

Der Compiler würde in diesem Fall eine Fehlermeldung ausgeben.

- Jede Anweisung muss durch ein Semikolon abgeschlossen werden, anders als etwa in Pascal, wo das Semikolon Anweisungen trennt.
- Die **return**-Anweisung beendet die Funktion und liefert 0 als Funktionswert an die rufende Funktion zurück. Für **main** ist die rufende Funktion das Betriebssystem.

Verschiedene Funktionswerte von **main** können verabredet werden, um dem Betriebssystem den korrekten (0) oder fehlerhaften (> 0) Ablauf eines Programms mitzuteilen, wobei unterschiedliche Werte zur Kennzeichnung unterschiedlicher Fehler verwendet werden können. Dass der Funktionswert von **main** ganzzahlig sein muss, ist durch den Typ **int** festgelegt.

Um das Programm auszuführen muss es noch compiliert und gelinkt werden. Die Vorgehensweise dazu hängt vom Compiler und Linker ab. Im Anhang A werden die benötigten Schritte für den GCC-Compiler unter UNIX erklärt. Für andere Compiler muss die entsprechende Dokumentation konsultiert werden.

1.5 Literatur

Dieser Text kann nur eine Einführung in die Programmierung mit ANSI-C geben und es bleiben viele interessante und nützliche Aspekte unerwähnt. Ein sehr gutes deutschsprachiges Lehrbuch ist [9]. Leider ist es beim Verlag vergriffen und kann derzeit nur antiquarisch erworben oder in Bibliotheken ausgeliehen werden. Auch [3] ist empfehlenswert. Der Klassiker [7] ist auch in deutscher Fassung erhältlich. Das Buch ist sicher lehrreich, kann aber wegen des dort verwendeten (zwar auch standardisierten aber weniger Typsicherheit bietenden) „C-Dialekts“ bei Anfängern einige Verwirrung stiften. Der C-Standard wird immer noch weiter entwickelt. Die aktuelle Entwurfsfassung ist derzeit unter [6] auch im Internet verfügbar, ältere Dokumente zu Standardisierungsvarianten sind über [5] zugänglich. Sehr nützlich ist das Buch [4], das den C-Standard in kommentierender Form beschreibt.

Für genauere Informationen zur Philosophie und auch zur Entstehungs- und Standardisierungsgeschichte der Programmiersprache kann der derzeitige Wikipedia-Eintrag [1] (und ggf. neuere Versionen) empfohlen werden. Weitere verlässliche und empfehlenswerte Internet-Quellen sind die in [8] gesammelten Links (sehr lehrreich ist zum Beispiel die dort erwähnte FAQ-Liste zur C-Programmierung, die in erweiterter Fassung auch als Buch erschienen ist). Diese Webseite wurde in der Newsgruppe `comp.lang.c` eine Zeit lang als nützlichste Website zur C-Programmierung betrachtet.

Zu jeder C-Implementation gehört eine Dokumentation. Diese Dokumentation ist dafür die verlässlichste Informationsquelle, auch wenn es sich um eine Implementation des ANSI-C-Standards handeln soll. Es kann z.B. Abweichungen in der mitgelieferten Standardbibliothek geben, Implementationsfehler sind möglich (sofern sie bekannt sind, wird eine gute Dokumentation sie erwähnen) und die meisten Compiler bieten in Details Erweiterungen, die über die Forderungen des Standards hinausgehen. Wer möglichst portabel programmieren will oder muss, ist allerdings gut beraten, auf die Ausnutzung solcher Erweiterungen weitestgehend zu verzichten. Auf Linux-Systemen ist normalerweise die sogenannte GNU Compiler Collection (GCC) installiert. Dies ist eigentlich eine Sammlung von Programmierwerkzeugen für C und weitere Programmiersprachen — trotzdem sprechen wir GCC hier meist als „den Compiler“ an mit dem Aufruf `gcc`. Zusammen mit der GNU C-Library `glibc` in der jeweiligen Version ist auch noch eine umfangreiche Dokumentation installierbar, die auch Auskunft über verfügbare Programmierwerkzeuge (wie zum Beispiel das in Kapitel 8 erwähnte `make`) gibt. Im Internet ist diese Dokumentation (und auch der Compiler selbst) unter [2] verfügbar.

